

Arquitetura e Estrutura de Pastas

Simulador Estrategico de Loja | Squad 16

Monorepo | Node/Express + React/Vite | Prisma ORM

Versao 1.0 | Abril 2026

Este documento descreve a arquitetura geral do projeto, a estrutura de pastas do monorepo, os padroes arquiteturais adotados, o fluxo de dados entre camadas e as estrategias para lidar com latencia, erros e indisponibilidade. O projeto nao integra servicos de IA externos nesta versao.

1. Stack Tecnologico Completo

1.1 Backend

Runtime	Node.js v22.14.0	Ambiente de execucao JavaScript server-side
Framework	Express.js v5.2.1	Servidor HTTP, roteamento, middlewares
ORM	Prisma v5.22.0	Acesso ao banco, migrations, schema tipado
BD Producao	PostgreSQL v14+	Banco relacional principal
BD Local/Dev	SQLite	Banco local via schema.local.prisma + local.db
Autenticacao	jsonwebtoken v9.0.3	Geracao e verificacao de JWT (8h expiry)
Criptografia	bcryptjs v3.0.3	Hash de senhas (salt rounds: 10)
Validacao	Zod v4.3.6	Schemas de validacao de entrada
CORS	cors v2.8.6	Controle de origem cruzada
Env vars	dotenv v17.3.1 + dotenv-cli v7.4.4	Variaveis de ambiente por arquivo
Dev server	nodemon v3.1.14	Reinicio automatico em desenvolvimento
Testes	Jest v30.3.0	Framework de testes unitarios
HTTP Testing	Supertest v7.2.2	Testes de integracao HTTP
Mock	jest-mock-extended v4.0.0	Mocks tipados para dependencias
Modulos	CommonJS (require/module.exports)	Padrao de modulos do Node.js

1.2 Frontend

Build Tool	Vite v8.0.1	Bundler e dev server ultra-rapido
Framework UI	React v19.2.4	Biblioteca de interface com hooks
Roteamento	React Router DOM v7.13.2	Navegacao client-side (SPA)
Estado Global	Zustand v5.0.12	Store leve com persist no localStorage
Forms	React Hook Form v7.72.0	Gerenciamento de formularios
Form Validation	@hookform/resolvers v5.2.2	Integracao Zod + React Hook Form
Validacao	Zod v4.3.6	Schemas compartilhados front/back
HTTP Client	Axios v1.14.0	Requisicoes HTTP com interceptors
Graficos	Recharts v3.8.1	Graficos de barras/linhas para resultados

CSS	Tailwind CSS v4.2.2	Utilitarios CSS via @tailwindcss/vite
Lint	ESLint v9.39.4	Qualidade e padronizacao de codigo
PostCSS	PostCSS v8.5.8 + Autoprefixer v10.	Processamento CSS
Tipos	@types/react v19.2.14	Suporte a tipos TS/IntelliSense
Modulos	ESM (import/export)	Padrao de modulos ECMAScript

1.3 Ferramentas de IA

Sem Ferramentas de IA Integradas aoCodigo

A aplicacao NAO integra nenhuma ferramenta de inteligencia artificial paga ou gratuita (OpenAI, Anthropic Claude, Google Gemini, Hugging Face, Cohere, etc.). Todo o processamento de DRE, feedbacks e ranking e realizado por algoritmos deterministicos escritos em JavaScript puro, sem chamadas a APIs externas de IA.

O Claude Code (Anthropic) foi utilizado como ferramenta de auxilio no desenvolvimento (geracao e revisao de codigo, documentacao), mas NÃO e integrado ao codigo da aplicacao.

2. Estrutura de Pastas — Monorepo

Organizacao de alto nivel do repositorio:

/	Raiz do monorepo
backend/	API REST Node.js/Express
frontend/	SPA React/Vite
docs/	Documentacoes e PDFs
.claude/	Configuracoes Claude Code (nao vai para producao)

2.1 Backend — Estrutura Detalhada

backend/	Raiz do backend
src/	Codigo-fonte principal
server.js	Ponto de entrada, configura Express e rotas
controllers/	Handlers HTTP, validacao, resposta
authController.js	Login/JWT
userController.js	CRUD de usuarios
productController.js	CRUD de produtos
squadController.js	CRUD de squads + membros
storeController.js	Criacao/consulta de lojas
inventoryController.js	Gestao de estoque
roundController.js	Criacao/encerramento de rodadas
simulationController.js	Submit config, preview DRE, resultados
services/	Logica de negocio pura
financeService.js	Motor de calculo DRE + feedbacks
simulationService.js	Processamento da rodada (transacoes)
rankingService.js	Ordenacao e formatacao do ranking
repositories/	Acesso ao banco via Prisma
userRepository.js	Queries de usuarios
productRepository.js	Queries de produtos

squadRepository.js	Queries de squads
storeRepository.js	Queries de lojas
inventoryRepository.js	Queries de estoque
roundRepository.js	Queries de rodadas
roundConfigRepository.js	Queries de configuracoes de rodada
routes/	Definicao de rotas Express
middlewares/	authMiddleware, roleMiddleware, errorMiddleware
utils/asyncHandler.js	Wrapper para async/await nos controllers
prisma/	ORM e schema do banco
schema.prisma	Schema principal (PostgreSQL)
schema.local.prisma	Schema local (SQLite)
seed.js	Dados iniciais (users, squads, produtos)
prisma/local.db	Arquivo SQLite local
.env / .env.local	Variaveis de ambiente (nunca commitadas)
jest.config.js	Configuracao dos testes
coverage/	Relatorio HTML de cobertura de testes

2.2 Frontend — Estrutura Detalhada

frontend/	Raiz do frontend
src/	Codigo-fonte principal
main.jsx	Entry point React, renderiza App
App.jsx	BrowserRouter, definicao de todas as rotas
pages/	Paginas da aplicacao (1 por rota)
LoginPage.jsx	Formulario de login
StoresDashboardPage.jsx	Painel do Player (visao da loja)
RoundConfigPage.jsx	Configuracao de estrategia da rodada
ResultsPage.jsx	Resultado financeiro do Player
RankingPage.jsx	Ranking por rodada
AdminDashboardPage.jsx	Painel do Game Master
RoundsManagementPage.jsx	Gestao de rodadas (GM)
SquadsManagementPage.jsx	Gestao de squads e usuarios (GM)
ProductsManagementPage.jsx	Catalogo de produtos (GM)
AdminResultsPage.jsx	Resultados de todos os squads (GM)
UnauthorizedPage.jsx	Pagina de acesso negado
components/	Componentes reutilizaveis
layout/	PlayerLayout, AdminLayout, PrivateRoute
ui/	Button, Input, Toast, Skeleton, ErrorMessage
DREPreview.jsx	Preview do DRE (grafico + feedbacks)
services/	Chamadas HTTP (uma por recurso)
api.js	Instancia Axios + interceptors
authService.js	Login
roundService.js	Rodadas + config + preview
storeService.js	Lojas + inventario

productService.js	Produtos
squadService.js	Squads
userService.js	Usuarios
store/	Estado global Zustand
authStore.js	Token + user + isAuthenticated (persist)
hooks/useToast.js	Hook para notificacoes toast
utils/formatters.js	Formatacao de moeda, datas, etc.
public/	Assets publicos (favicon, icons)
dist/	Build de producao (gerado pelo Vite)
vite.config.js	Configuracao do Vite
eslint.config.js	Configuracao do ESLint

3. Arquitetura do Backend — Camadas

O backend segue uma arquitetura em camadas (Layered Architecture) com separacao clara de responsabilidades:

HTTP Layer — Routes + Middlewares
Express Router: define verbos HTTP e paths
authMiddleware: valida JWT em todas as rotas privadas
roleMiddleware: verifica permissao por role (GAME_MASTER / PLAYER)
errorMiddleware: captura todas as excecoes nao tratadas
Controller Layer — src/controllers/
Recebe req/res do Express
Valida entrada com Zod (safeParse)
Orquestra chamadas a Services e Repositories
Retorna resposta HTTP com status code semantico
Encapsulado por asyncHandler para propagacao de erros
Service Layer — src/services/
financeService.js: calculo DRE (effectiveVolume, grossRevenue, netProfit, netMargin)
simulationService.js: processamento da rodada com Prisma \$transaction
rankingService.js: ordenacao por netMargin e netProfit
Repository Layer — src/repositories/
Encapsula todas as queries Prisma
Uma funcao por operacao (findById, create, update, remove, etc.)
Nenhuma logica de negocio – apenas acesso a dados
Data Layer — Prisma ORM + Banco de Dados
PostgreSQL (producao) / SQLite (dev local)
Transacoes atomicas via prisma.\$transaction()
Schema versionado em prisma/schema.prisma

4. Arquitetura do Frontend

4.1 Fluxo de Dados

Fluxo de uma requisicao:

```
Componente/Pagina
-> React Hook Form + Zod (validacao local)
-> services/xxx.js (chamada Axios)
    -> api.js interceptor.request (injeta Bearer token)
-> Backend API (HTTP)
    <- api.js interceptor.response (trata 401 -> logout)
<- Promise resolve/reject
-> Estado local (useState) ou Zustand store
-> Re-render da UI
```

4.2 Estado Global (Zustand + localStorage)

```
// store/authStore.js
const useAuthStore = create(
  persist(
    (set) => ({
      user: null, token: null, isAuthenticated: false,
      login: (user, token) => set({ user, token, isAuthenticated: true }),
      logout: () => set({ user: null, token: null, isAuthenticated: false }),
    }),
    { name: "simulador-auth" } // key no localStorage
  )
);
```

4.3 Protecao de Rotas (PrivateRoute)

```
// Rota privada so para PLAYER
<PrivateRoute allowedRoles={["PLAYER"]} >
  <RoundConfigPage />
</PrivateRoute>

// PrivateRoute verifica:
// 1. isAuthenticated -> se nao: redireciona para /login
// 2. user.role in allowedRoles -> se nao: redireciona para /unauthorized
```

4.4 Integracao Frontend <-> Backend

Nao ha integracao com APIs de IA. A unica integracao externa do frontend e com o proprio backend (REST API em localhost:3333 ou URL configurada via VITE_API_URL).

```
// api.js - configuracao base
const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL || "http://localhost:3333",
});

// Request interceptor: injeta token JWT automaticamente
api.interceptors.request.use((config) => {
  const token = useAuthStore.getState().token;
```

```
    if (token) config.headers.Authorization = `Bearer ${token}`;
    return config;
  });

// Response interceptor: trata 401 (sessao expirada)
api.interceptors.response.use(
  (res) => res,
  (err) => {
    if (err.response?.status === 401) {
      useAuthStore.getState().logout();
      window.location.href = "/login";
    }
    return Promise.reject(err);
  }
);
```

5. Tratamento de Latencia, Erros, Timeout e Indisponibilidade

5.1 Estados de Carregamento (Loading States)

Todas as paginas com carregamento assincrono implementam estados visuais para feedback ao usuario enquanto aguardam respostas da API:

```
// Padrao aplicado em todas as paginas que carregam dados
const [loading, setLoading] = useState(true);

// Enquanto carrega: exhibe skeletons animados
if (loading) return (
  <Layout>
    <Skeleton variant="card" className="h-28" />
    <Skeleton variant="table" rows={5} />
  </Layout>
);
```

- Skeleton loaders previnem Cumulative Layout Shift (CLS)
- Botoes com prop loading={isSubmitting} ficam desabilitados durante submit
- Impede duplo-clique e envios duplicados de formulario

5.2 Tratamento de Falhas de Carregamento (loadError + Retry)

```
// Falha ao carregar dados iniciais
const [loadError, setLoadError] = useState("");

async function load() {
  setLoadError(""); // limpa erro anterior antes de tentar
  try {
    const data = await api.get(...);
  } catch {
    setLoadError("Nao foi possivel carregar os dados.");
  } finally {
    setLoading(false);
  }
}
```

```
// Exibe ErrorMessage com botao "Tentar novamente"
if (loadError) return (
  <ErrorMessage message={loadError} onRetry={() => { setLoading(true); load(); }} />
);
```

5.3 Falhas em Acoes (Mutations) — Toast de Erro

```
// Acoes de escrita (submit, create, delete, restock)
try {
  await service.create(data);
  toast.success("Operacao realizada com sucesso!");
} catch (err) {
  // Prioriza mensagem do backend; fallback para mensagem generica
  const msg = err.response?.data?.message
    ?? err.response?.data?.error
    ?? "Erro inesperado. Tente novamente.";
  toast.error(msg);
}
```

5.4 Timeout — Ausencia de Configuracao Explicita

Timeout Padrao do Axios

O Axios nao tem timeout configurado explicitamente no projeto (valor padrao: 0 = sem timeout). Em producao, recomenda-se configurar: `api.defaults.timeout = 10000`; (10 segundos) para evitar requisicoes penduradas. O navegador tem seu proprio timeout (~2min), mas a UX fica ruim.

5.5 Indisponibilidade do Backend

Cenario	Comportamento Atual	Resultado para o Usuario
Backend offline (ECONNREFUSED)	Axios lanca <code>NetworkError</code>	catch captura, toast.error ou loadError exibido
Backend lento (>10s sem timeout)	Spinner infinito na UI	Recomendado: adicionar timeout ao Axios
BD offline (Prisma connection)	Erro 500 do backend	errorMiddleware retorna 500; frontend exibe toast
Token expirado mid-session	Backend retorna 401	Interceptor Axios: logout + redirect /login
Rodada fechada durante config	Backend retorna 409	toast.error com mensagem do backend

5.6 Resiliencia no Backend — Transacoes Atomicas

O processamento de rodadas usa Prisma `$transaction` para garantir atomicidade. Falhas em uma loja nao afetam as demais (try/catch por loja, rollback automatico).

```
// simulationService.js
for (const config of configs) {
  try {
    await prisma.$transaction(async (tx) => { /* ... */ });
  } catch (err) {
    // Rollback automatico. Continua para proxima loja.
    console.error(`Erro loja ${config.storeId}:`, err.message);
  }
}
```

6. Modelo de Dados — Resumo das Entidades Prisma

Entidade	Campos Principais
User	id, name, email, password (hash), role, leader, squadId
Squad	id, name -> Users[]
Store	id, name, squadId -> Inventory[], RoundConfig[]
Product	id, name, purchasePrice
Inventory	storeId + productId (PK composta), quantity
Round	id, number, status (OPEN/PROCESSING/CLOSED), startDate, endDate
RoundConfig	id, roundId, storeId, fixedExpenses, variableExpenses, submittedAt
RoundConfigItem	id, roundConfigId, productId, salePrice, salesVolume
FinancialResult	id, roundId, storeId, grossRevenue, costs, expenses, grossProfit, netProfit, netMargin

7. Recomendacoes de Melhoria para Producao

Timeout Axios	Configurar timeout de 10-15s para evitar requisicoes penduradas
Rate Limiting	Adicionar express-rate-limit para proteger /auth/login contra brute force
Helmet.js	Adicionar helmet ao Express para cabecalhos de seguranca HTTP
Logs estruturados	Substituir console.error por winston ou pino com nivel e timestamp
Health check avancado	Endpoint /health verificar conexao com BD (prisma.\$queryRaw)
Variavel VITE_API_URL	Configurar em producao para URL real do backend (nao localhost)
HTTPS	Servir frontend e backend via HTTPS em producao
Retry com backoff	Para futuras integracoes externas, implementar retry exponencial